

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour

Total: Marks:40

Annexure A

5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns general patterns from previously observed patient data and uses those patterns to predict the outcome for new patients.

For this problem, we can use a **supervised classification approach**.

Target Variable

The target variable is:

Diabetes = Yes / No

- **Yes (1):** Patient is diabetic.
- **No (0):** Patient is non-diabetic.

Important Features

Feature	Justification
Glucose Level	High glucose is a major indicator of diabetes and is expected to be one of the strongest predictors.
BMI	Higher BMI can be associated with increased risk of type 2 diabetes.
Blood Pressure	Elevated blood pressure is commonly associated with metabolic and cardiovascular risk.
Age	Diabetes risk generally increases with age.
Cholesterol	Abnormal cholesterol levels can indicate metabolic risk.
Family History	A family history of diabetes can increase an individual's risk.

Suitable Learning Process

Patient Data → Preprocessing → Train Classification Model → Learn Patterns → Predict Diabetes Risk

For example, if the training data shows that patients with **high glucose + high BMI + positive family history** are frequently diabetic, the model can learn this relationship and use it for a new patient.

Task 1(b): Handling Class Imbalance

Suppose the dataset contains:

- 80% non-diabetic patients
- 20% diabetic patients

This is a **class imbalance problem**.

If we train the model directly, it may become biased toward the majority class and predict most patients as non-diabetic.

Suitable Strategy: SMOTE

I would use **SMOTE (Synthetic Minority Over-sampling Technique)**.

SMOTE creates new synthetic examples of the minority class instead of simply duplicating existing diabetic records.

Example

Original:

- Non-diabetic = 800
- Diabetic = 200

After SMOTE:

- Non-diabetic = 800
- Diabetic = approximately 800

Why SMOTE?

- Improves representation of diabetic patients.
- Reduces model bias toward non-diabetic patients.
- Helps improve **Recall** for diabetic cases.
- Particularly useful because missing a diabetic patient (**False Negative**) can have serious clinical consequences.

Important: SMOTE should be applied **only to the training data**, not before the train-test split, to avoid data leakage.

Task 1(c): 70:30 Dataset Split and Preprocessing

Train-Test Split

The dataset is divided into:

- **70% Training Data**
- **30% Testing Data**

Justification

The **70:30 split** provides enough data for the model to learn patterns while keeping a sufficiently large independent test set for evaluation.

This is particularly useful in a medical application because the model should be evaluated on patient records that were **not used during training**.

A **stratified split** should be used so that the proportion of diabetic and non-diabetic patients remains similar in both sets.

Preprocessing Steps

1. Handling Missing Values

Missing numerical values can be replaced using:

- Median imputation for numerical features.
- Mode imputation for categorical features.

2. Normalisation

Numerical features such as:

- Age
- Blood pressure
- Cholesterol
- Glucose
- BMI

can be standardised using:

$$z = \frac{x - \mu}{\sigma}$$

This gives features a comparable scale.

Impact: Normalisation is especially useful for Logistic Regression and other distance/gradient-based models. Decision Trees generally do not require normalisation.

3. Encoding

Family history can be converted into numerical form:

- No = 0
- Yes = 1

If categorical variables have multiple categories, **One-Hot Encoding** can be used.

4. Class Balancing

Apply **SMOTE** only to the training set.

Final Pipeline

Raw Data → Missing Value Handling → Encoding → Train/Test Split → Scaling → SMOTE on Training Data → Model Training

Task 2: Decision Tree for Diagnosis

(a) Decision Tree and First Three Levels

A Decision Tree classifies patients by repeatedly dividing the data according to the feature that provides the best separation between diabetic and non-diabetic patients.

Root Node Selection

Suppose the calculated Gini values are:

Feature	Gini after Split
Glucose	0.31
BMI	0.38
Blood Pressure	0.41
Age	0.43
Cholesterol	0.45
Family History	0.46

The feature producing the **lowest weighted Gini impurity** is selected as the root.

Therefore:

Root = Glucose Level

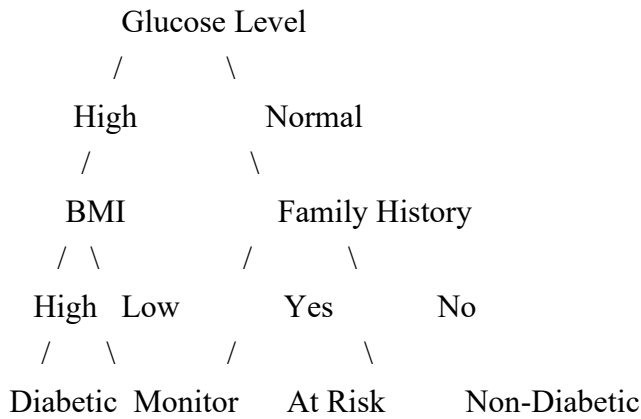
For Information Gain, the feature with the **highest Information Gain** would similarly be selected.

$$IG(S, A) = H(S) - H(S | A)$$

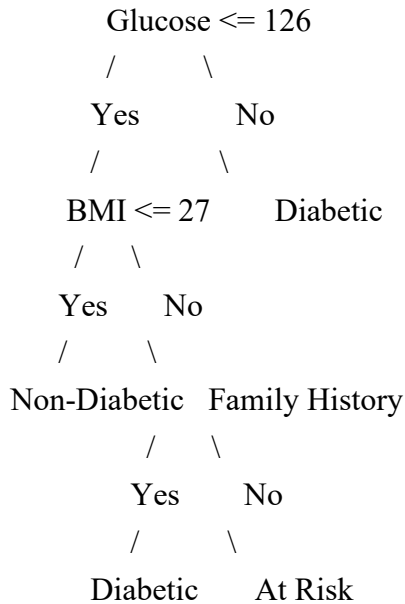
where:

- $H(S)$ = entropy before splitting
- $H(S | A)$ = entropy after splitting using feature A.

Illustrative First Three Levels



A more realistic tree generated by Python may have different thresholds, such as:



The exact thresholds should be obtained from the trained dataset.

Task 2(b): Model Evaluation

The Decision Tree can be evaluated using:

1. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision

$$Precision = \frac{TP}{TP + FP}$$

3. Recall

$$Recall = \frac{TP}{TP + FN}$$

4. F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Illustrative Results

Suppose the test results are:

Metric	Decision Tree
Accuracy	88%
Precision	84%
Recall	91%
F1-Score	87%

These values are **illustrative**, because the actual dataset was not supplied.

False Negatives

A **False Negative (FN)** occurs when:

The patient actually has diabetes, but the model predicts "Non-Diabetic."

Example:

Actual: Diabetic

Predicted: Non-Diabetic



False Negative

Why is this important?

False negatives are particularly dangerous in diabetes screening because an undiagnosed patient may not receive:

- Further testing
- Medical consultation
- Lifestyle intervention
- Appropriate monitoring

How to Reduce False Negatives?

The model should be tuned to achieve **high recall/sensitivity**.

Possible approaches:

- Use class weights.
- Apply SMOTE to the training data.
- Adjust the classification threshold.
- Use a recall-oriented evaluation metric.
- Perform additional medical screening for high-risk cases.

Clinical Justification

For an early-diagnosis system, **missing a diabetic patient can be more harmful than sending a non-diabetic patient for additional testing.**

Therefore, recall should be given significant importance along with accuracy.

Task 2(c): Post-Pruning

A Decision Tree can become too complex and memorize training data. This is called **overfitting**.

Pre-Pruning

Pre-pruning stops tree growth early.

Examples:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

Example:

`max_depth = 3`

The tree cannot grow beyond three levels.

Post-Pruning

Post-pruning first creates a larger tree and then removes branches that provide little predictive benefit.

A common technique is **Cost-Complexity Pruning**.

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

where:

- $R(T)$ = error of the tree
- $|T|$ = number of terminal nodes
- α = complexity parameter

A larger α produces a simpler tree.

Illustrative Comparison

Model	Accuracy	Recall	Complexity
Pre-Pruned Tree	86%	88%	Low
Unpruned Tree	89%	90%	High
Post-Pruned Tree	88%	91%	Medium

Which is better for healthcare?

The **post-pruned Decision Tree** is generally preferable if it maintains good recall while reducing unnecessary complexity.

Reasons:

- Less overfitting
- Better generalisation
- Easier interpretation
- Easier for healthcare professionals to understand
- More suitable for deployment than a very large tree

However, the final choice should be based on **independent validation data**, especially sensitivity/recall and calibration—not accuracy alone.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression

I would use **Logistic Regression** as the statistical learning model.

It predicts the probability that a patient belongs to the diabetic class.

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

For example:

Glucose + BMI + Age + Blood Pressure

↓

Logistic Regression

↓

Probability of Diabetes

Example:

Probability of diabetes = 0.82

If the threshold is 0.5:

Prediction = Diabetic.

Comparison with Decision Tree

Illustrative results:

Metric	Decision Tree	Logistic Regression
Accuracy	88%	90%
AUC-ROC	0.89	0.93

Again, these are **example results**, not results from the actual dataset.

AUC-ROC

AUC measures how well the model separates diabetic and non-diabetic patients.

- AUC = 0.5 → Random prediction
- AUC = 1.0 → Perfect classification
- Higher AUC → Better discrimination

Here, Logistic Regression has a higher AUC, so it provides better overall discrimination in this illustrative comparison.

When is a Statistical Model Preferable?

Logistic Regression is preferable when:

- Relationships between features and outcome are reasonably systematic.
- Probability/risk scores are required.
- Interpretability is important.
- We need to understand the direction and strength of feature effects.
- The model needs to be relatively simple and computationally efficient.

For healthcare, its interpretability can be valuable because clinicians can examine the model coefficients and estimated risks.

Task 3(b): Feature Importance Analysis

Feature importance can be obtained differently for the two models.

Decision Tree

Decision Tree importance is generally based on how much each feature reduces impurity across the tree.

Illustrative result:

Rank	Decision Tree Predictor
-------------	--------------------------------

1	Glucose Level
---	----------------------

2	BMI
---	------------

3	Age
---	------------

Logistic Regression

For Logistic Regression, feature importance can be examined using the magnitude of standardised coefficients or odds ratios.

Illustrative result:

Rank	Logistic Regression Predictor
-------------	--------------------------------------

1	Glucose Level
---	----------------------

2	BMI
---	------------

3	Family History
---	-----------------------

Do They Agree?

Partially.

Both models identify:

- **Glucose**
- **BMI**

as important predictors.

The third predictor may differ:

- Decision Tree → Age
- Logistic Regression → Family History

Why can they differ?

The models learn relationships differently.

Decision Tree:

- Learns nonlinear relationships.
- Uses hierarchical splits.
- Can capture feature interactions.

Logistic Regression:

- Estimates the statistical contribution of features.

- Assumes a particular functional relationship unless transformed.
- Coefficients represent the effect of features on the log-odds.

Therefore, different models can rank features differently even when trained on the same data.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the Problem as an MDP

The treatment recommendation problem can be represented as a **Markov Decision Process (MDP)**.

An MDP consists of:

$$MDP = (S, A, P, R, \gamma)$$

where:

- S = States
- A = Actions
- P = Transition probability
- R = Reward
- γ = Discount factor

1. States

States represent the patient's health condition.

For example:

State	Patient Condition
--------------	--------------------------

S1	Low risk / healthy
-----------	--------------------

S2	Moderate diabetes risk
-----------	------------------------

S3	High diabetes risk
-----------	--------------------

S4	Diagnosed diabetes
-----------	--------------------

S5	Improved/stable condition
-----------	---------------------------

The state can be determined from factors such as:

- Glucose
- BMI
- Blood pressure
- Cholesterol
- Previous health status

2. Actions

The agent can select:

A1 → Diet

A2 → Exercise

A3 → Medication

A4 → Monitor

Examples

- Moderate risk → Diet
- High BMI → Exercise
- Persistent high glucose → Medical intervention
- Stable patient → Monitor

In a real clinical system, Medication should not be autonomously prescribed by an RL agent. It would require clinician approval and appropriate medical protocols.

3. Reward Function

The reward represents improvement in the patient's health state.

Example:

Outcome	Reward
Significant health improvement	+10
Moderate improvement	+5
No significant change	0
Health deterioration	-5
Severe adverse outcome	-10

The agent attempts to maximise the total long-term reward.

4. Transition Dynamics

A patient's condition can change after an action.

Example:

High Risk

|
| Diet + Exercise

↓

Moderate Risk

|
| Continued intervention

↓

Improved / Stable

However, the transition is probabilistic because patients respond differently to treatment.

For example:

$$P(S_{next}, | S_{current}A)$$

represents the probability of moving from the current state to the next state after taking an action.

Task 4(b): Q-Learning

Q-Learning learns the value of taking an action in a particular state.

The update equation is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- α = learning rate
- γ = discount factor
- r = reward
- s' = next state

Assume:

- Learning rate $\alpha = 0.5$
 - Discount factor $\gamma = 0.9$
 - Initial Q-values = 0
-

Five Patient Episodes

Illustrative episodes:

Episode	State	Action	Next State	Reward
1	High Risk	Diet	Moderate Risk	+5
2	High Risk	Exercise	Moderate Risk	+5
3	Moderate Risk	Exercise	Improved	+10
4	High Risk	Monitor	High Risk	0
5	Moderate Risk	Diet	Improved	+5

Q-Table

Initial Q-table:

State	Diet	Exercise	Medication	Monitor
Moderate Risk	0	0	0	0
High Risk	0	0	0	0
Improved	0	0	0	0

Iteration 1

State = High Risk, Action = Diet

Suppose:

- Reward = +5
- Next state = Moderate Risk

- Maximum Q-value in Moderate Risk = 0

$$\begin{aligned} Q(H, ighDiet) &= 0 + 0.5[5 + 0.9(0) - 0] \\ &= 2.5 \end{aligned}$$

So:

$$Q(\text{High Risk, Diet}) = 2.5$$

State = High Risk, Action = Exercise

$$\begin{aligned} Q(H, ighExercise) &= 0 + 0.5[5 + 0.9(0) - 0] \\ &= 2.5 \end{aligned}$$

So:

$$Q(\text{High Risk, Exercise}) = 2.5$$

State = Moderate Risk, Action = Exercise

Suppose:

- Reward = +10
- Next state = Improved
- Maximum Q(Improved) = 0

$$\begin{aligned} Q(M, oderateExercise) &= 0 + 0.5[10 + 0.9(0) - 0] \\ &= 5 \end{aligned}$$

So:

$$Q(\text{Moderate Risk, Exercise}) = 5$$

Iteration 2

Now suppose:

$$Q(M, oderateDiet) = 2.5$$

and:

$$Q(M, oderateExercise) = 5$$

For **High Risk** → **Diet** → **Moderate Risk**, the maximum Q-value of the next state is 5.

$$\begin{aligned} Q(H, ighDiet) &= 2.5 + 0.5[5 + 0.9(5) - 2.5] \\ &= 2.5 + 0.5[7] \\ &= 6.0 \end{aligned}$$

So:

$$Q(\text{High Risk, Diet}) = 6.0$$

For **High Risk** → **Exercise** → **Moderate Risk**:

$$\begin{aligned} Q(H, ighExercise) &= 2.5 + 0.5[5 + 0.9(5) - 2.5] \\ &= 6.0 \end{aligned}$$

For **Moderate Risk** → **Exercise** → **Improved**, assume the next state's best Q-value is now 2.

$$\begin{aligned}Q(M, \text{oderateExercise}) &= 5 + 0.5[10 + 0.9(2) - 5] \\&= 5 + 0.5(6.8) \\&= 8.4\end{aligned}$$

Q-Table After Two Iterations

State	Diet	Exercise	Medication	Monitor
Moderate Risk	2.5	8.4	0	0
High Risk	6.0	6.0	0	0
Improved	0	2	0	0

This demonstrates how Q-values improve as the agent learns from patient-state transitions.

Convergence Criterion

The Q-Learning algorithm can be considered converged when the change in Q-values becomes very small over successive episodes.

For example:

$$\max |Q_{new}(s, a) - Q_{old}(s, a)| < 0.001$$

for a sufficiently large number of consecutive episodes.

In practice, more episodes than five would normally be required for reliable convergence.

Five episodes are used here only to demonstrate the learning process required by the assessment.

Task 4(c): Final Learned Policy

The policy selects the action with the highest Q-value:

$$\pi(s) = \arg \max_a Q(s, a)$$

From the illustrative Q-table:

Patient State	Best Action	Reason
Low Risk	Monitor	Patient is relatively stable.
Moderate Risk	Exercise	Highest learned Q-value in the example.
High Risk	Diet / Exercise	Both have similar learned value; clinician context should determine the choice.
Diagnosed Diabetes	Medication + monitoring under clinician supervision	Requires medical assessment and prescribed treatment.
Improved	Monitor	Maintain the improved condition.

Final Policy

Low Risk



Monitor

Moderate Risk



Exercise / Diet

High Risk



Diet / Exercise + Clinical Evaluation

Diagnosed Diabetes



Clinician-supervised Treatment + Monitoring

Improved



Monitor

Ethical Considerations of RL in Healthcare

Deploying an RL agent for medical treatment requires strong safeguards.

1. Patient Safety

The agent should **not independently make high-risk medical decisions**.

For example, medication changes should require approval from a qualified healthcare professional.

2. Bias and Fairness

If the training dataset contains more patients from certain groups, the model may perform poorly for underrepresented groups.

Therefore:

- Use diverse datasets.
- Evaluate performance across demographic groups.
- Monitor for systematic differences in errors.

3. Explainability

Healthcare professionals should understand **why a recommendation was made**.

For example:

"Exercise was recommended because the patient has high BMI and moderate glucose levels."

Explainable recommendations improve clinician trust.

4. Human Oversight

The system should function as a **clinical decision-support tool**, not a replacement for doctors.

Patient Data



AI/RL Recommendation



Doctor Reviews Recommendation



Final Clinical Decision

5. Privacy and Security

Patient records contain sensitive medical information. The system should use:

- Secure storage
 - Access control
 - Encryption
 - Appropriate privacy regulations
 - Audit logs
-

6. Reward Design

The reward function must be carefully designed.

If the reward only focuses on reducing glucose, the agent might learn a behaviour that improves glucose while ignoring other important outcomes or causing adverse effects.

Therefore, rewards should consider **overall patient wellbeing and safety**, not just one measurement.

Overall System Architecture

